

High Level Design Document (HLD)

1. System Overview

This system is an AI powered media processing microservice. It allows registered users to upload images and automatically extracts useful information from those images using artificial intelligence models.

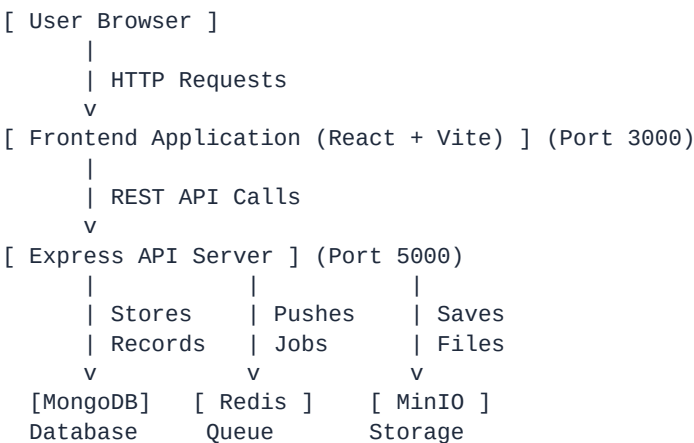
The system processes image files asynchronously in the background. When a user uploads an image file, the system immediately accepts the file and gives the user a unique job tracking code. The user does not need to wait on the screen while the AI models analyze the image. The background worker picks up the file from a queue, runs the AI analysis, and saves the final enriched results into the database.

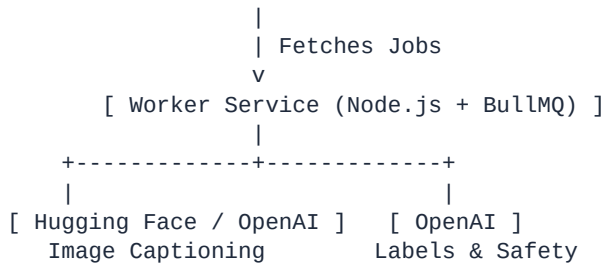
2. Business Objectives and Requirements

- **User Account Management:** Users can create accounts and log in securely.
- **File Upload Validation:** Accepts image files in JPG, PNG, and WEBP formats with a maximum file size limit of 5 megabytes.
- **Background Queue Processing:** Image processing runs asynchronously in the background using a message queue.
- **AI Feature Extraction:**
 1. Image Captioning: Generates a clear natural language description of the image.
 2. Object Detection: Identifies key objects and concepts in the image with score values.
 3. Content Safety Verification: Checks if the image contains adult, violent, medical, spoof, or racy content.
 - **Flagged Content Alerts:** Automatically flags unsafe images and creates in-app user notifications.
 - **Job Tracking and Manual Retry:** Users can view past processing jobs, check detailed results, and manually retry any job that failed.

3. High Level System Architecture

The application is built using a decoupled microservice architecture:





4. Main System Components

- **Frontend Web Client:** Built with React and Vite. It provides simple screens for user registration, login, image uploading, job status checking, detailed result views, and user notifications.
- **Express API Server:** A Node.js web server that receives user requests, checks authentication, validates file uploads, saves files to storage, creates job records in the database, and adds job tasks to the queue.
- **Worker Service:** A separate Node.js background process powered by BullMQ. It listens to the Redis queue, downloads image files from storage, calls external AI models, updates job states in the database, and creates notifications if content is flagged.
- **MongoDB Database:** The database used to hold user accounts, job processing records, AI analysis outputs, and user notification messages.
- **Redis Queue Broker:** An in-memory data store used by BullMQ to manage background job queues, concurrency, and automatic retries.
- **MinIO File Storage:** An S3 compatible local file storage service used to store uploaded image files safely.
- **External AI Models:**
 - * Hugging Face Inference API (Salesforce BLIP model) for image captioning.
 - * OpenAI Vision (gpt-4o-mini) as a backup for image captioning and primary provider for object label detection.
 - * OpenAI Moderation API (omni-moderation-latest) for content safety checking.

5. End to End Data Flow

1. **Upload Request:** The user selects an image file in the browser and sends it to the API server.
2. **Validation and Storage:** The API server checks the file type and file size. It uploads the image buffer to MinIO storage.
3. **Job Creation:** The API server creates a job record in MongoDB with a status of pending and pushes a new task into the Redis BullMQ queue.
4. **Immediate Response:** The API server returns the job ID and pending status to the user immediately.
5. **Background Processing:** The Worker service picks up the task from the queue, updates the job status to processing, and downloads the file from storage.
6. **AI Execution:** The Worker runs three AI tasks in order: image captioning, label detection, and safety verification.
7. **Result Persistence:** The Worker saves the caption, detected labels, safety scores, and completion status to MongoDB.
8. **Notification Handling:** If the safety check marks the image as flagged, the Worker creates a notification record for the user.
9. **UI Update:** The React frontend polls the API server for job updates and displays the final results to the user.

6. Security and Authentication Strategy

- **Authentication:** Uses JSON Web Tokens (JWT). Short term access tokens last 15 minutes and stay in memory. Long term refresh tokens last 7 days and are saved in secure HTTP-only cookies.
- **Route Protection:** All API endpoints except login, signup, and health check require valid Bearer token authentication.
- **File Upload Protections:** File size limits (5 megabytes) and allowed image types (JPG, PNG, WEBP) are strictly enforced at the server layer.

7. Deployment and Cloud Architecture

- **Containerization:** Every service (API, Worker, Frontend) includes a Dockerfile.
- **Local Setup:** Managed via Docker Compose for easy single-command startup (``docker compose up --build``).
- **Cloud Platform:** Configured for Render cloud platform using a ``render.yaml`` blueprint file and automated GitHub Actions CI/CD deployment hooks.